

INF781

SEGURIDAD DE SOFTWARE

GUÍA DE LABORATORIO

Autenticación de Dos Factores

(2FA) con TOTP en Laravel 13

Construcción del proyecto INF781-Laravel2FA desde cero

Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Carrera	Ingeniería Informática
Universidad	Universidad Autónoma Tomás Frías — Potosí, Bolivia
Stack tecnológico	Laravel 13 · PHP 8.3+ · PostgreSQL · Breeze · Google2FA
Repositorio de referencia	github.com/HuascarFedor/INF781-Laravel2FA
Duración estimada	5 horas de laboratorio

COMPETENCIA DE LA GUÍA

Implementar autenticación de dos factores (2FA) basada en contraseñas de un solo uso (TOTP, RFC 6238) sobre una aplicación Laravel 13 con base de datos PostgreSQL, integrando Google Authenticator mediante la librería pragmarx/google2fa-laravel y la generación de códigos QR con bacon/bacon-qr-code, aplicando los principios de defensa en profundidad y autenticación multifactor estudiados en clase.

1. Introducción y marco teórico

1.1 ¿Qué es la autenticación de dos factores?

La autenticación de dos factores (2FA, del inglés Two-Factor Authentication) es un mecanismo de seguridad que exige al usuario presentar dos pruebas independientes de identidad antes de obtener acceso a un sistema. Esto reduce drásticamente el riesgo de accesos no autorizados aun cuando la contraseña haya sido comprometida (por phishing, fugas de bases de datos o ataques de fuerza bruta).

Los factores de autenticación se clasifican tradicionalmente en tres categorías:

Factor	Categoría	Ejemplo
Algo que sabes	Conocimiento (knowledge)	Contraseña, PIN, respuesta secreta
Algo que tienes	Posesión (possession)	Smartphone con app TOTP, llave U2F, token físico
Algo que eres	Inherencia (inherence)	Huella digital, rostro, voz, iris

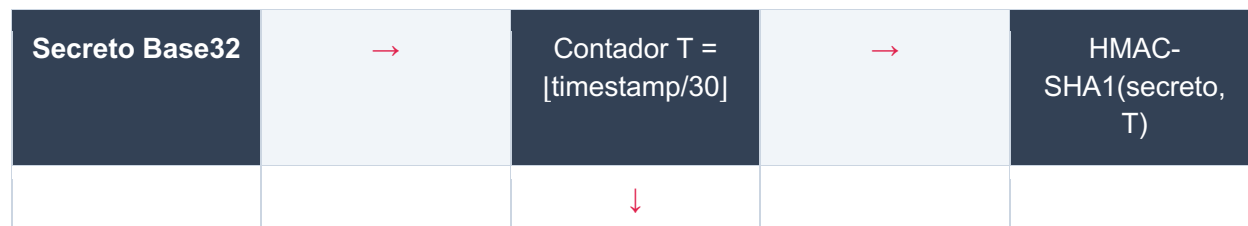
En este laboratorio

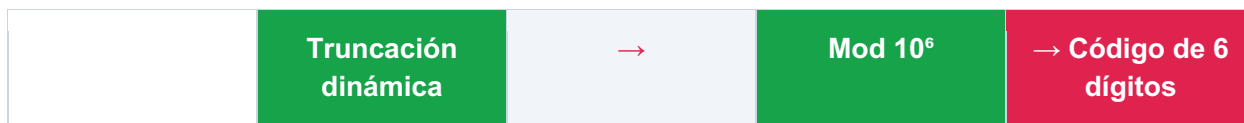
La aplicación combina factor de conocimiento (la contraseña almacenada con bcrypt) con factor de posesión (un código TOTP generado por una app autenticadora instalada en el smartphone del usuario). El código se renueva cada 30 segundos y nunca se transmite por la red al servidor — el servidor lo recalcula localmente con el secreto compartido.

1.2 TOTP: Time-based One-Time Password (RFC 6238)

El estándar TOTP, definido en el RFC 6238, especifica un algoritmo para generar contraseñas de un solo uso basadas en el tiempo. La idea central es que tanto el servidor como el cliente comparten un secreto criptográfico aleatorio (generalmente codificado en Base32) durante la fase de configuración (típicamente escaneando un código QR). A partir de ese secreto, ambos pueden derivar de forma independiente un código numérico de 6 dígitos que cambia cada 30 segundos.

El cálculo simplificado es:





Las propiedades de seguridad clave del TOTP son:

- Cada código es válido únicamente durante una ventana de 30 segundos.
- El secreto compartido se transmite una sola vez (durante el escaneo del QR) y nunca más sale del dispositivo.
- El cliente y el servidor calculan el código de manera offline, sin tráfico de red durante el login.
- Es resistente a ataques de phishing simples y a la reutilización de credenciales filtradas.

1.3 Componentes del proyecto INF781-Laravel2FA

La aplicación que construirás en esta guía implementa el flujo completo de 2FA TOTP sobre un proyecto Laravel 13 con autenticación basada en Breeze. Los componentes principales son:

Componente	Responsabilidad
<code>TwoFactorController</code>	Generación del secreto, render del QR de configuración, activación/desactivación del 2FA por el propio usuario.
<code>TwoFactorVerifyController</code>	Pantalla intermedia post-login que recibe el código TOTP de 6 dígitos, lo valida y marca la sesión como verificada.
<code>TwoFactorMiddleware</code>	Intercepta cada request: si el usuario tiene 2FA activado pero la sesión todavía no fue verificada, lo redirige al formulario de verificación.
Migración 2FA	Agrega dos columnas a <code>users</code> : <code>two_factor_secret</code> y <code>two_factor_enabled</code> .
Vistas Blade	<code>setup.blade.php</code> (configuración con QR) y <code>verify.blade.php</code> (challenge post-login).
PostgreSQL	Motor de base de datos para usuarios, sesiones y password resets. La sesión se almacena en la tabla <code>sessions</code> porque <code>SESSION_DRIVER=database</code> .

1.4 Flujo de autenticación con 2FA

La siguiente secuencia describe el flujo completo desde el inicio de sesión hasta el acceso al dashboard cuando el usuario tiene 2FA habilitado:

- 1 El usuario envía email + contraseña al endpoint POST `/login`.

2	Laravel valida las credenciales contra la tabla <code>users</code> (verifica el hash bcrypt de password). Si son correctas, crea la sesión.
3	El usuario es redirigido a <code>/dashboard</code> . El middleware <code>two-factor</code> detecta que <code>two_factor_enabled = true</code> y que la sesión no contiene la marca <code>two_factor_verified</code> .
4	El middleware redirige a <code>/two-factor/verify</code> . Se muestra un formulario que pide el código de 6 dígitos.
5	El servidor recibe el código y lo verifica con <code>Google2FA::verifyKey()</code> contra el secreto guardado en la columna <code>two_factor_secret</code> .
6	Si el código es válido, se guarda <code>session(['two_factor_verified' => true])</code> y el usuario es redirigido al dashboard.

1.5 Objetivos de aprendizaje

Al finalizar esta guía de laboratorio, el estudiante será capaz de:

- Configurar un proyecto Laravel 13 con base de datos PostgreSQL desde cero, incluyendo Breeze para autenticación.
- Instalar e integrar las librerías `pragmarx/google2fa-laravel` y `bacon/bacon-qr-code`.
- Crear migraciones que extiendan la tabla `users` con columnas para almacenar el secreto y el estado del 2FA.
- Implementar controladores para la activación, verificación y desactivación de 2FA.
- Construir un middleware personalizado que aplique la verificación 2FA a las rutas protegidas.
- Generar códigos QR en formato SVG para la configuración del segundo factor.
- Validar el flujo completo end-to-end con Google Authenticator o una app TOTP equivalente.

2. Requisitos previos

Antes de comenzar el laboratorio, asegúrate de tener instalado y funcional el siguiente software en tu equipo:

Software	Versión mínima	Comando de verificación
PHP	8.3+	<code>php --version</code>
Composer	2.x	<code>composer --version</code>
Node.js	20.x LTS	<code>node --version</code>
NPM	10.x	<code>npm --version</code>
PostgreSQL	15+	<code>psql --version</code>
Git	2.x	<code>git --version</code>
Extensiones PHP	—	<code>pdo_pgsql, mbstring, xml, gd. Verifica con <code>php -m grep pgsql</code></code>

2.1 App autenticadora en el smartphone

Para esta guía necesitarás también una aplicación TOTP instalada en tu smartphone. Cualquiera de estas opciones es válida (todas implementan RFC 6238):

- Google Authenticator (Android / iOS) — la más común, sin nube.
- Microsoft Authenticator (Android / iOS) — soporta backup en la nube.
- Authy (Android / iOS / Desktop) — sincronización multi-dispositivo.
- FreeOTP (Android / iOS) — open source, sin telemetría.

TIP — Si no tienes smartphone

Puedes usar una extensión de navegador como "Authenticator" para Chrome/Firefox, o aplicaciones de escritorio como KeePassXC. Cualquier herramienta que escanee un QR `otpauth://` y genere códigos de 6 dígitos cada 30 segundos es compatible.

3. Procedimiento paso a paso

Convención

Cada paso siguiente parte del estado dejado por el anterior. No saltes pasos. Si en algún punto el comportamiento de tu aplicación no coincide con lo esperado, retrocede al paso previo y verifica el estado antes de continuar.

PASO 1 Crear el proyecto Laravel 13

Abre una terminal en el directorio donde guardas tus proyectos académicos (por ejemplo, ~/Proyectos/INF781) y crea un nuevo proyecto Laravel con Composer. El nombre del proyecto será INF781-Laravel2FA.

```
Terminal  bash

# Crear el proyecto desde cero
composer create-project laravel/laravel:^13.0 INF781-Laravel2FA

# Entrar al directorio del proyecto
cd INF781-Laravel2FA

# Verificar que la versión instalada es Laravel 13
php artisan --version
```

La salida del último comando debe ser similar a:

```
Laravel Framework 13.x.y
```

CAPTURA 01

Proyecto Laravel recién creado

Captura del terminal mostrando la salida de "php artisan --version" con la versión 13.x

PHP versión mínima

Laravel 13 requiere PHP 8.3 o superior. Si Composer falla con un mensaje sobre la versión de PHP, actualiza tu instalación antes de continuar. En macOS con Homebrew: `brew install php@8.3 && brew link php@8.3 --overwrite --force`.

PASO 2 Crear la base de datos PostgreSQL

Conéctate al servidor PostgreSQL como superusuario (postgres) y crea la base de datos junto con un usuario dedicado para la aplicación. Esto sigue el principio de mínimo privilegio: la app no debe usar el rol postgres.

```
Terminal — psql  sql

# Conectarte al servidor PostgreSQL
```

```
psql -U postgres

-- Una vez dentro de psql, ejecuta:
CREATE USER laravel_2fa_user WITH PASSWORD 'secret2fa';
CREATE DATABASE laravel_2fa OWNER laravel_2fa_user;
GRANT ALL PRIVILEGES ON DATABASE laravel_2fa TO laravel_2fa_user;

-- Verificar que se creó:
\l laravel_2fa

-- Salir
\q
```

CAPTURA 02

Base de datos PostgreSQL creada

Captura del psql mostrando la creación del usuario, la base de datos y la salida de \l laravel_2fa

En macOS con Postgres.app

Si usas Postgres.app, abre la terminal integrada de la app (haciendo clic en el icono → "Open psql") y omite el "psql -U postgres" inicial. Postgres.app usa tu usuario macOS por defecto, que ya es superusuario.

PASO 3 Configurar las variables de entorno

Abre el archivo `.env` en la raíz del proyecto y modifica las variables de la sección `DB_*` para que apunten a la base PostgreSQL recién creada. También cambia `SESSION_DRIVER` a `database` para que las sesiones se persistan en PostgreSQL.

```
.env  env

APP_NAME="INF781 2FA"
APP_ENV=local
APP_KEY=
APP_DEBUG=true
APP_URL=http://localhost:8000

# Base de datos PostgreSQL
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=laravel_2fa
DB_USERNAME=laravel_2fa_user
DB_PASSWORD=secret2fa

# Sesiones en base de datos (necesario para flujo 2FA estable)
SESSION_DRIVER=database
SESSION_LIFETIME=120

# Mantener cache y queue en database para simplificar
CACHE_STORE=database
```

```
QUEUE_CONNECTION=database

# Mailer en log durante desarrollo (no envía correos reales)
MAIL_MAILER=log
```

A continuación, regenera la APP_KEY y verifica que la conexión a PostgreSQL funciona ejecutando las migraciones base de Laravel:

```
Terminal  bash

# Generar la clave de aplicación
php artisan key:generate

# Verificar que la conexión a PostgreSQL funciona
php artisan migrate

# Deberías ver algo como:
# Migration table created successfully.
# 0001_01_01_000000_create_users_table ..... DONE
# 0001_01_01_000001_create_cache_table ..... DONE
# 0001_01_01_000002_create_jobs_table ..... DONE
```

CAPTURA 03

Migraciones iniciales aplicadas en PostgreSQL

Captura del terminal mostrando "Migration table created successfully" y las 3 migraciones DONE

Si la migración falla

Errores comunes: (a) "could not connect to server" → PostgreSQL no está corriendo. (b) "password authentication failed" → revisa DB_USERNAME / DB_PASSWORD. (c) "database does not exist" → el nombre en DB_DATABASE no coincide con el que creaste. (d) "could not find driver" → falta la extensión pdo_pgsql; instálala y reinicia tu terminal.

PASO 4 Instalar Laravel Breeze (autenticación base)

Laravel Breeze proporciona un sistema de autenticación mínimo (login, registro, recuperación de contraseña, perfil de usuario) con vistas Blade y Tailwind CSS. Lo instalaremos para no escribir el flujo de login desde cero — sobre Breeze añadiremos la capa de 2FA.

```
Terminal  bash

# Instalar Breeze como dependencia de desarrollo
composer require laravel/breeze --dev

# Ejecutar el instalador con stack Blade
php artisan breeze:install blade

# Cuando pregunte "¿Quieres ejecutar pnpm/npm install y npm run build?",
# responde "Yes" o ejecuta manualmente:
npm install
```



```
npm run build
```

Breeze creó automáticamente:

- Las rutas de autenticación en `routes/auth.php`.
- Los controladores en `app/Http/Controllers/Auth/`.
- Las vistas Blade en `resources/views/auth/` y los layouts en `resources/views/layouts/`.
- El controlador `ProfileController` y la vista `dashboard.blade.php`.

Verifica que la instalación funciona levantando el servidor:

```
Terminal  bash

php artisan serve

# En otra terminal (opcional, para hot reload de assets):
npm run dev
```

Abre `http://localhost:8000` en el navegador. Verás la página welcome con enlaces "Log in" y "Register" en la esquina superior derecha. Crea una cuenta de prueba con tu email y una contraseña; serás redirigido al dashboard.

CAPTURA 04

Registro y dashboard de Breeze funcionando

Captura mostrando el dashboard "You're logged in!" después del registro exitoso

PASO 5 Instalar las librerías de 2FA

Necesitamos dos paquetes de Composer:

- `pragmarx/google2fa-laravel` — implementación TOTP compatible con Google Authenticator (genera secretos, valida códigos OTP).
- `bacon/bacon-qr-code` — generador de códigos QR en SVG sin dependencias nativas (no necesita extensión imagick).

```
Terminal  bash

# Instalar ambos paquetes en una sola línea
composer require pragmarx/google2fa-laravel:^3.0 bacon/bacon-qr-code:^3.1

# Publicar el archivo de configuración de google2fa
php artisan vendor:publish --provider="PragmaRX\Google2FALaravel\ServiceProvider"
```

El comando `vendor:publish` crea el archivo `config/google2fa.php` con los parámetros del paquete. Para esta guía aceptamos los valores por defecto — los inspeccionaremos brevemente.

Sobre la configuración de Google2FA

El parámetro más importante es `window` (por defecto 1). Define cuántas ventanas de 30 s antes y después de la actual se aceptan como válidas, para tolerar pequeños desfases de reloj entre cliente y servidor. Un valor de 1 acepta el código actual y los dos adyacentes.

PASO 6 Crear la migración para las columnas 2FA

Necesitamos extender la tabla `users` con dos columnas:

- `two_factor_secret` (string, nullable) — almacena el secreto Base32 generado por Google2FA.
- `two_factor_enabled` (boolean, default false) — indica si el usuario activó 2FA en su cuenta.

Genera el archivo de migración con artisan:

```
Terminal  bash

php artisan make:migration add_two_factor_to_users_table --table=users
```

Edita el archivo recién creado en `database/migrations/` (su nombre incluye un timestamp) y reemplaza el contenido por:

```
database/migrations/YYYY_MM_DD_HHMMSS_add_two_factor_to_users_table.php  php

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('two_factor_secret')->nullable()->after('password');
            $table->boolean('two_factor_enabled')->default(false)-
>after('two_factor_secret');
        });
    }

    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn(['two_factor_secret', 'two_factor_enabled']);
        });
    }
};
```

Ejecuta la migración:

```
Terminal  bash
```

```
php artisan migrate
```

Verifica en PostgreSQL que las columnas se agregaron correctamente:

```
Terminal — psql  sql

psql -U laravel_2fa_user -d laravel_2fa -h 127.0.0.1

-- Una vez dentro:
\d users

-- Deberías ver las columnas two_factor_secret y two_factor_enabled
\q
```

CAPTURA 05

Estructura de la tabla users con columnas 2FA

Captura del psql mostrando la salida de \d users con two_factor_secret y two_factor_enabled visibles

PASO 7 Actualizar el modelo User

El modelo `app/Models/User.php` debe declarar los nuevos campos como fillable, ocultar el secreto en serializaciones y castear `two_factor_enabled` como boolean. Reemplaza el archivo completo por:

```
app/Models/User.php  php

<?php

namespace App\Models;

use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Attributes\Fillable;
use Illuminate\Database\Eloquent\Attributes\Hidden;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

#[Fillable(['name', 'email', 'password', 'two_factor_secret',
'two_factor_enabled'])]
#[Hidden(['password', 'remember_token', 'two_factor_secret'])]
class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable;

    /**
     * Los atributos que deben ser casteados.
     *
     * @return array<string, string>
     */
    protected function casts(): array
```

```
{
    return [
        'email_verified_at' => 'datetime',
        'password' => 'hashed',
        'two_factor_enabled' => 'boolean',
    ];
}
```

Sobre los atributos PHP 8

Laravel 13 introdujo los atributos `#[Fillable]` y `#[Hidden]` como alternativa a las propiedades `$fillable` y `$hidden`. La propiedad `#[Hidden]` que incluye `two_factor_secret` garantiza que el secreto nunca aparezca en respuestas JSON (`toArray` / `toJson`) — un detalle de seguridad importante.

PASO 8

Crear el TwoFactorController (configuración del 2FA)

Este controlador maneja la página donde el usuario activa o desactiva el 2FA en su propia cuenta. Tiene tres acciones:

- `show()` — genera el secreto si no existe, calcula el QR en SVG y muestra la vista de configuración.
- `enable()` — recibe el código TOTP, lo verifica y, si es correcto, marca `two_factor_enabled = true`.
- `disable()` — desactiva el 2FA y borra el secreto de la base de datos.

Crea el controlador:

Terminal `bash`

```
php artisan make:controller TwoFactorController
```

Reemplaza el contenido del archivo generado por:

app/Http/Controllers/TwoFactorController.php `php`

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use PragmaRX\Google2FA\Google2FA;
use BaconQrCode\Renderer\ImageRenderer;
use BaconQrCode\Renderer\Image\SvgImageBackEnd;
use BaconQrCode\Renderer\RendererStyle\RendererStyle;
use BaconQrCode\Writer;

class TwoFactorController extends Controller
{
    // Muestra la página de configuración 2FA con QR
    public function show(Request $request)
```

```
{
    $user = $request->user();
    $google2fa = new Google2FA();

    // Si el usuario aún no tiene secreto, lo generamos y guardamos
    if (!$user->two_factor_secret) {
        $secret = $google2fa->generateSecretKey();
        $user->update(['two_factor_secret' => $secret]);
    }

    // Construimos la URL otpauth:// que el QR codifica
    $qrCodeUrl = $google2fa->getQRCodeUrl(
        config('app.name'),
        $user->email,
        $user->two_factor_secret
    );

    // Renderizamos el QR como SVG (sin dependencias nativas)
    $renderer = new ImageRenderer(
        new RendererStyle(200),
        new SvgImageBackEnd()
    );
    $writer = new Writer($renderer);
    $qrCodeSvg = $writer->writeString($qrCodeUrl);

    return view('two-factor.setup', [
        'qrCodeSvg' => $qrCodeSvg,
        'secret' => $user->two_factor_secret,
        'enabled' => $user->two_factor_enabled,
    ]);
}

// Activa 2FA después de verificar el código
public function enable(Request $request)
{
    $request->validate(['code' => 'required|string']);

    $user = $request->user();
    $google2fa = new Google2FA();

    $valid = $google2fa->verifyKey($user->two_factor_secret, $request->code);

    if (!$valid) {
        return back()->withErrors(['code' => 'El código OTP es inválido.']);
    }

    $user->update(['two_factor_enabled' => true]);

    return redirect()->route('two-factor.setup')
        ->with('status', '2FA activado correctamente.');
```

```
}

// Desactiva 2FA
public function disable(Request $request)
```

```
{
    $request->user()->update([
        'two_factor_enabled' => false,
        'two_factor_secret' => null,
    ]);

    return redirect()->route('two-factor.setup')
        ->with('status', '2FA desactivado.');
```

PASO 9 Crear el TwoFactorVerifyController (challenge post-login)

Este segundo controlador maneja la pantalla intermedia que aparece justo después del login cuando el usuario tiene 2FA activado. Tiene dos acciones:

- `show()` — muestra el formulario para ingresar el código de 6 dígitos.
- `verify()` — valida el código y marca la sesión como `two_factor_verified`.

Crea el controlador:

```
Terminal  bash

php artisan make:controller TwoFactorVerifyController
```

Reemplaza el contenido del archivo generado por:

```
app/Http/Controllers/TwoFactorVerifyController.php  php

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use PragmaRX\Google2FA\Google2FA;

class TwoFactorVerifyController extends Controller
{
    // Muestra el formulario de verificación OTP post-login
    public function show()
    {
        return view('two-factor.verify');
    }

    // Verifica el código OTP y completa el login
    public function verify(Request $request)
    {
        $request->validate(['code' => 'required|string']);

        $user = $request->user();
        $google2fa = new Google2FA();

        $valid = $google2fa->verifyKey($user->two_factor_secret, $request->code);
```

```
        if (!$valid) {
            return back()->withErrors(['code' => 'Código OTP inválido. Intenta de nuevo.']);
        }

        // Marcar la sesión como verificada con 2FA
        $request->session()->put('two_factor_verified', true);

        return redirect()->intended(route('dashboard'));
    }
}
```

PASO 10 Crear el middleware TwoFactorMiddleware

El middleware es la pieza clave del flujo: intercepta cada request a las rutas protegidas y, si el usuario tiene 2FA activado pero todavía no validó el código en esta sesión, lo redirige al formulario de verificación.

Crea el middleware:

```
Terminal  bash

php artisan make:middleware TwoFactorMiddleware
```

Reemplaza el archivo generado por:

```
app/Http/Middleware/TwoFactorMiddleware.php  php

<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class TwoFactorMiddleware
{
    /**
     * Handle an incoming request.
     *
     * @param Closure(Request): (Response) $next
     */
    public function handle(Request $request, Closure $next): Response
    {
        $user = $request->user();

        if ($user && $user->two_factor_enabled && !$request->session()->get('two_factor_verified')) {
            // Permitir acceso a las rutas de verificación 2FA sin loop infinito
            if (!$request->routeIs('two-factor.verify') && !$request->routeIs('two-factor.verify.post')) {
```

```
        return redirect()->route('two-factor.verify');
    }

    return $next($request);
}
}
```

Por qué la condición de exclusión

Si no excluyéramos las rutas `two-factor.verify` y `two-factor.verify.post`, el middleware redirigiría a `/two-factor/verify` y de ahí intentaría redirigir de nuevo a `/two-factor/verify`, generando un loop infinito (HTTP 310 ERR_TOO_MANY_REDIRECTS).

Ahora registra el middleware con un alias en `bootstrap/app.php`. En Laravel 11+ el archivo `bootstrap/app.php` reemplaza al antiguo `Kernel.php`. Modifica la sección `withMiddleware`:

```
bootstrap/app.php  php

<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->alias([
            'two-factor' => \App\Http\Middleware\TwoFactorMiddleware::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```

PASO 11 Definir las rutas web

Edita `routes/web.php` para agregar las rutas de 2FA y proteger el dashboard. Reemplaza el contenido por:

```
routes/web.php  php

<?php

use App\Http\Controllers\ProfileController;
use App\Http\Controllers\TwoFactorController;
use App\Http\Controllers\TwoFactorVerifyController;
use Illuminate\Support\Facades\Route;
```



```
Route::get('/', function () {
    return view('welcome');
});

// Ruta de verificación 2FA (solo auth, sin two-factor middleware para evitar
loop)
Route::middleware('auth')->group(function () {
    Route::get('/two-factor/verify', [TwoFactorVerifyController::class, 'show'])
        ->name('two-factor.verify');
    Route::post('/two-factor/verify', [TwoFactorVerifyController::class,
'verify'])
        ->name('two-factor.verify.post');
});

// Rutas protegidas: requieren auth + verificación 2FA
Route::middleware(['auth', 'verified', 'two-factor'])->group(function () {
    Route::get('/dashboard', function () {
        return view('dashboard');
    })->name('dashboard');

    Route::get('/profile', [ProfileController::class, 'edit'])->
>name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->
>name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->
>name('profile.destroy');

    // Gestión de 2FA
    Route::get('/two-factor/setup', [TwoFactorController::class, 'show'])
        ->name('two-factor.setup');
    Route::post('/two-factor/enable', [TwoFactorController::class, 'enable'])
        ->name('two-factor.enable');
    Route::post('/two-factor/disable', [TwoFactorController::class, 'disable'])
        ->name('two-factor.disable');
});

require __DIR__.'/auth.php';
```

Estructura de los grupos de rutas

Existen DOS grupos: el primero solo requiere "auth" (para que el usuario pueda llegar al formulario de verificación). El segundo requiere "auth + verified + two-factor" (todo lo demás de la app). El middleware "two-factor" no se aplica al primer grupo porque entraría en loop.

Verifica que las rutas se registraron correctamente:

Terminal bash

```
php artisan route:list | grep two-factor

# Deberías ver:
# GET    /two-factor/verify ..... two-factor.verify
# POST   /two-factor/verify ..... two-factor.verify.post
```

```
# GET    /two-factor/setup ..... two-factor.setup
# POST   /two-factor/enable ..... two-factor.enable
# POST   /two-factor/disable ..... two-factor.disable
```

CAPTURA 06

Rutas de 2FA registradas

Captura del terminal con la salida de "php artisan route:list | grep two-factor"

PASO 12 Crear las vistas Blade

Necesitamos dos vistas. Crea el directorio `resources/views/two-factor/` con dos archivos.

```
Terminal  bash

mkdir -p resources/views/two-factor
```

12.1 Vista de configuración (setup.blade.php)

Esta vista muestra el QR cuando el 2FA está desactivado, y un botón para desactivar cuando ya está activado.

```
resources/views/two-factor/setup.blade.php  blade

<x-app-layout>
  <x-slot name="header">
    <h2 class="font-semibold text-xl text-gray-800 leading-tight">
      Autenticación en Dos Factores (2FA)
    </h2>
  </x-slot>

  <div class="py-12">
    <div class="max-w-2xl mx-auto sm:px-6 lg:px-8">
      <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg p-6">

        @if (session('status'))
          <div class="mb-4 p-4 bg-green-100 text-green-800 rounded">
            {{ session('status') }}
          </div>
        @endif

        @if ($enabled)
          <div class="mb-6 p-4 bg-green-50 border border-green-300
rounded">
            <p class="text-green-700 font-semibold">
              2FA está ACTIVADO en tu cuenta.
            </p>
          </div>

          <form method="POST" action="{{ route('two-factor.disable') }}">
            @csrf
            <button type="submit">
```

```

        class="bg-red-600 hover:bg-red-700 text-white font-
        bold py-2 px-4 rounded"
        onclick="return confirm('¿Deseas desactivar el
        2FA?')">
            Desactivar 2FA
        </button>
    </form>
    @else
    <p class="mb-4 text-gray-700">
        Escanea este código QR con tu app autenticadora
        (Google Authenticator, Authy, etc.) y luego ingresa
        el código de 6 dígitos para activar 2FA.
    </p>

    <div class="flex justify-center mb-4">
        {!! $qrCodeSvg !!}
    </div>

    <div class="mb-6">
        <p class="text-sm text-gray-500">
            O ingresa manualmente esta clave secreta:
        </p>
        <code class="block bg-gray-100 p-2 rounded text-sm mt-1
        tracking-widest">
            {{ $secret }}
        </code>
    </div>

    <form method="POST" action="{{ route('two-factor.enable') }}">
        @csrf
        <div class="mb-4">
            <label class="block text-sm font-medium text-gray-700
            mb-1">
                Código OTP de verificación
            </label>
            <input type="text" name="code" maxlength="6"
            autocomplete="one-time-code"
            class="border border-gray-300 rounded px-3 py-2 w-
            40 text-center tracking-widest text-lg
            @error('code') border-red-500 @enderror"
            placeholder="000000">
            @error('code')
            <p class="text-red-600 text-sm mt-1">{{ $message
            }}</p>
            @enderror
        </div>
        <button type="submit"
            class="bg-blue-600 hover:bg-blue-700 text-white font-
            bold py-2 px-4 rounded">
            Activar 2FA
        </button>
    </form>
    @endif
</div>
</div>
</div>

```

```
</x-app-layout>
```

Sobre el helper {!! !!}

En Blade, `{{ $var }}` escapa el HTML (anti-XSS). Para imprimir el SVG del QR usamos `{!! $qrCodeSvg !!}` que NO escapa. Esto es seguro aquí porque el SVG lo genera el servidor a partir de un secreto que no contiene input del usuario. NUNCA uses `{!! !!}` con datos provenientes del usuario sin sanitizar.

12.2 Vista de verificación (verify.blade.php)

Esta vista usa el layout de "guest" (sin navegación) para que el usuario solo pueda ingresar el código antes de continuar:

```
resources/views/two-factor/verify.blade.php  blade

<x-guest-layout>
    <div class="mb-4 text-sm text-gray-600">
        Tu cuenta tiene activada la autenticación en dos factores.
        Ingresa el código de 6 dígitos de tu app autenticadora para continuar.
    </div>

    <form method="POST" action="{{ route('two-factor.verify.post') }}">
        @csrf

        <div class="mb-4">
            <x-input-label for="code" value="Código OTP" />
            <x-text-input
                id="code"
                name="code"
                type="text"
                maxlength="6"
                autocomplete="one-time-code"
                class="block mt-1 w-full text-center tracking-widest text-lg"
                placeholder="000000"
                autofocus
            />
            <x-input-error :messages="$errors->get('code')" class="mt-2" />
        </div>

        <div class="flex items-center justify-end mt-4">
            <x-primary-button>
                Verificar
            </x-primary-button>
        </div>
    </form>
</x-guest-layout>
```

PASO 13 Agregar enlace al setup en el dashboard

Edita `resources/views/dashboard.blade.php` para añadir un badge con el estado del 2FA y un enlace a la página de configuración:

```
resources/views/dashboard.blade.php  blade

<x-app-layout>
  <x-slot name="header">
    <h2 class="font-semibold text-xl text-gray-800 leading-tight">
      {{ __('Dashboard') }}
    </h2>
  </x-slot>

  <div class="py-12">
    <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
      <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
        <div class="p-6 text-gray-900">
          {{ __('You're logged in!') }}
        </div>
        <div class="p-6 border-t border-gray-200">
          @if(auth()->user()->two_factor_enabled)
            <span class="inline-flex items-center px-3 py-1 rounded-
full text-sm font-medium bg-green-100 text-green-800 mr-3">
              2FA Activado
            </span>
          @else
            <span class="inline-flex items-center px-3 py-1 rounded-
full text-sm font-medium bg-yellow-100 text-yellow-800 mr-3">
              2FA Desactivado
            </span>
          @endif
          <a href="{{ route('two-factor.setup') }}"
            class="text-blue-600 hover:underline text-sm">
            Configurar Autenticación en Dos Factores
          </a>
        </div>
      </div>
    </div>
  </div>
</x-app-layout>
```

PASO 14 Pruebas end-to-end del flujo 2FA

Es momento de probar el flujo completo. Asegúrate de tener el servidor corriendo:

```
Terminal  bash

php artisan serve
```

14.1 Activación del 2FA

Sigue esta secuencia exactamente:

- Inicia sesión con la cuenta que creaste en el Paso 4. Llegas al dashboard con badge "2FA Desactivado".
- Haz clic en "Configurar Autenticación en Dos Factores". Verás la página de setup con un código QR.

- Abre Google Authenticator (o tu app TOTP) en el smartphone, presiona el botón "+", elige "Escanear código QR" y apunta la cámara a la pantalla.
- La app autenticadora añade una entrada con el nombre de la app y tu email; muestra un código de 6 dígitos que cambia cada 30 segundos.
- Ingresa el código actual en el campo "Código OTP de verificación" y haz clic en "Activar 2FA". Recibes un mensaje verde "2FA activado correctamente".

CAPTURA 07

Página de setup con QR generado

Captura de la página `/two-factor/setup` mostrando el QR, la clave secreta en texto plano y el campo OTP

CAPTURA 08

Google Authenticator escaneando el QR

Foto del smartphone con Google Authenticator mostrando la entrada recién agregada y el código de 6 dígitos vigente

CAPTURA 09

Mensaje de activación exitosa

Captura de la página de setup tras activar 2FA, mostrando el mensaje verde y el botón rojo "Desactivar 2FA"

14.2 Login con 2FA habilitado

Ahora prueba el flujo completo de autenticación con 2FA:

- Cierra sesión (botón "Log Out" en el menú del usuario).
- Vuelve a iniciar sesión con email y contraseña.
- En lugar del dashboard, eres redirigido a `/two-factor/verify` donde se te pide el código OTP.
- Mira tu app autenticadora, ingresa el código vigente y presiona "Verificar".
- Llegas al dashboard con el badge verde "2FA Activado".

CAPTURA 10

Pantalla intermedia de verificación 2FA

Captura de `/two-factor/verify` mostrando el formulario con el campo de código de 6 dígitos

CAPTURA 11

Dashboard con 2FA activado

Captura del dashboard mostrando el badge verde "2FA Activado"

14.3 Pruebas negativas

Verifica que el sistema rechaza correctamente entradas inválidas:

- **Código OTP inválido durante login** — ingresa un código aleatorio (ej. 000000). Recibes el error "Código OTP inválido. Intenta de nuevo." y permaneces en `/two-factor/verify`.
- **Acceso directo a ruta protegida** — sin completar el 2FA, intenta visitar `http://localhost:8000/dashboard` directamente. El middleware te redirige a `/two-factor/verify`.
- **Código expirado** — espera más de 60 s desde que la app generó un código y luego intenta usarlo. Debe ser rechazado (la ventana es ± 30 s con tolerancia ± 1).

CAPTURA 12

Error de código OTP inválido

Captura del formulario `/two-factor/verify` mostrando el mensaje de error en rojo "Código OTP inválido"

14.4 Verificación en la base de datos

Confirma con SQL que el secreto y el flag se guardaron correctamente:

```
Terminal — psql  sql

psql -U laravel_2fa_user -d laravel_2fa -h 127.0.0.1

-- Ver el estado del 2FA del usuario
SELECT id, name, email, two_factor_enabled,
       LENGTH(two_factor_secret) AS secret_length
FROM users;

-- El secret_length debe ser 16 (Base32 de 80 bits)
-- two_factor_enabled debe ser true

\q
```

Manejo del secreto en producción

En este laboratorio el `two_factor_secret` se guarda en texto plano por simplicidad pedagógica. En un entorno de producción real DEBES cifrar este campo en la base de datos. Laravel ofrece el cast "encrypted" — añade `'two_factor_secret' => 'encrypted'` en el método `casts()` del modelo `User`. Discutiremos esto en clase.

4. Lista de verificación

Antes de entregar la práctica, verifica cada uno de los siguientes puntos. Marca cada casilla únicamente cuando hayas comprobado el comportamiento en tu aplicación.

✓	#	Criterio
☐	1	El proyecto Laravel 13 fue creado con Composer y "php artisan --version" reporta 13.x.
☐	2	PostgreSQL contiene la base laravel_2fa propiedad del usuario laravel_2fa_user.
☐	3	El archivo .env apunta a la base PostgreSQL y SESSION_DRIVER=database.
☐	4	Laravel Breeze fue instalado con stack Blade y permite registrarse e iniciar sesión.
☐	5	Las librerías pragmarx/google2fa-laravel y bacon/bacon-qr-code están en composer.json.
☐	6	La migración add_two_factor_to_users_table fue ejecutada y agregó las dos columnas.
☐	7	El modelo User declara two_factor_secret y two_factor_enabled como fillable.
☐	8	El modelo User incluye two_factor_secret en los campos hidden (no se serializa).
☐	9	TwoFactorController genera secreto, renderiza QR en SVG y valida códigos OTP.
☐	10	TwoFactorVerifyController valida el código y guarda two_factor_verified en sesión.
☐	11	TwoFactorMiddleware redirige a /two-factor/verify cuando 2FA está habilitado.
☐	12	El alias "two-factor" está registrado en bootstrap/app.php.
☐	13	Las rutas web están organizadas en dos grupos: solo auth y auth+verified+two-factor.
☐	14	La vista setup.blade.php muestra el QR y la clave secreta en texto plano.
☐	15	La vista verify.blade.php usa x-guest-layout y tiene autofocus en el campo código.
☐	16	El dashboard muestra un badge con el estado del 2FA y un enlace al setup.
☐	17	Login + 2FA funciona end-to-end: credenciales → /two-factor/verify → /dashboard.
☐	18	Un código OTP inválido es rechazado con un mensaje de error claro.

✓	#	Criterio
☐	19	Visitar /dashboard sin verificar el 2FA redirige al formulario de verificación.
☐	20	No hay loop infinito al acceder a /two-factor/verify.

5. Rúbrica de evaluación

La práctica se evalúa sobre un total de 100 puntos. Los criterios se agrupan en seis dimensiones:

Criterio	Excelente	Aceptable	Insuficiente	Puntos
Configuración del proyecto y PostgreSQL	15	8	0	/ 15
Migración 2FA y modelo User actualizado	15	8	0	/ 15
Implementación de TwoFactorController (secreto + QR + activación)	20	10	0	/ 20
Middleware y challenge post-login funcionales	20	10	0	/ 20
Vistas Blade (setup + verify) con QR visible y formularios CSRF	15	8	0	/ 15
Pruebas end-to-end documentadas con capturas (1-12)	15	8	0	/ 15
TOTAL				/ 100

Bonificación opcional (+10 pts)

El estudiante que extienda la guía implementando códigos de respaldo (8 backup codes) hasheados con bcrypt, generados al activar 2FA, con flujo de login alternativo cuando se pierde el dispositivo) recibirá 10 puntos adicionales sobre la nota final, hasta un máximo de 100.

6. Entregables

Cada estudiante debe entregar los siguientes elementos a través del aula virtual del curso, en una única carpeta comprimida (ZIP) nombrada APELLIDO_NOMBRE_GL_2FA.zip:

6.1 Repositorio en GitHub

- Repositorio público con el código completo de la aplicación.
- Nombre sugerido: INF781-Laravel2FA-APELLIDO.
- README.md con: descripción del proyecto, requisitos, comandos de instalación (`composer install`, `php artisan migrate`, etc.) y datos de contacto.
- `.gitignore` que excluya `.env`, `vendor/` y `node_modules/`.
- Tag de versión `v1.0` apuntando al commit final del laboratorio.

6.2 Informe en PDF

- Documento PDF de 6 a 10 páginas con las 12 capturas listadas en la guía.
- Cada captura debe estar etiquetada (Captura 01, Captura 02, ...) con un breve comentario debajo.
- Incluir una sección de "Problemas encontrados y soluciones" donde el estudiante documente al menos un error que tuvo que depurar.

6.3 Estructura final esperada del proyecto

```
Estructura de archivos  tree
INF781-Laravel2FA/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Auth/
│   │   │   │   ├── # ← Generado por Breeze
│   │   │   │   ├── AuthenticatedSessionController.php
│   │   │   │   ├── RegisteredUserController.php
│   │   │   │   └── ... (otros)
│   │   │   ├── Controller.php
│   │   │   ├── ProfileController.php
│   │   │   ├── TwoFactorController.php ← NUEVO
│   │   │   └── TwoFactorVerifyController.php ← NUEVO
│   │   └── Middleware/
│   │       └── TwoFactorMiddleware.php ← NUEVO
│   └── Models/
│       └── User.php ← MODIFICADO (fillable + casts)
├── bootstrap/
│   └── app.php ← MODIFICADO (alias middleware)
├── config/
│   └── google2fa.php ← Publicado por vendor:publish
├── database/
│   └── migrations/
│       ├── 0001_01_01_000000_create_users_table.php
│       └── YYYY_MM_DD_HHMMSS_add_two_factor_to_users_table.php ← NUEVA
```

```
├── resources/
│   └── views/
│       ├── auth/
│       │   ├── dashboard.blade.php
│       │   ├── layouts/
│       │   └── two-factor/
│       │       ├── setup.blade.php
│       │       └── verify.blade.php
│       └── # ← Generado por Breeze
│           ├── MODIFICADO (badge 2FA)
│           ├── # ← Generado por Breeze
│           └── ← NUEVO
├── routes/
│   ├── auth.php
│   └── web.php
│       ├── # ← Generado por Breeze
│       └── ← MODIFICADO
├── .env.example
├── composer.json
├── google2fa + bacon-qr-code
└── README.md
```

6.4 Fecha y forma de entrega

La fecha límite y el medio de entrega serán comunicados por el docente en el aula virtual. Las entregas tardías se penalizan con 10 puntos por día de retraso, hasta un máximo de 3 días.

7. Referencias bibliográficas

- M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). TOTP: Time-Based One-Time Password Algorithm. RFC 6238. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc6238>
- NIST Special Publication 800-63B (2024). Digital Identity Guidelines: Authentication and Lifecycle Management. National Institute of Standards and Technology. <https://pages.nist.gov/800-63-3/sp800-63b.html>
- Otwell, T. y comunidad Laravel (2025). Laravel 13 Documentation: Authentication. <https://laravel.com/docs/13.x/authentication>
- Otwell, T. y comunidad Laravel (2025). Laravel 13 Documentation: Middleware. <https://laravel.com/docs/13.x/middleware>
- Antonio Carlos Ribeiro (2024). pragmarx/google2fa-laravel: Google Two-Factor Authentication for Laravel. <https://github.com/antonioribeiro/google2fa-laravel>
- Bacon QR Code Project (2024). bacon/bacon-qr-code: QR Code generator for PHP. <https://github.com/Bacon/BaconQrCode>
- OWASP Foundation (2025). Multi-Factor Authentication Cheat Sheet. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html
- PostgreSQL Global Development Group (2025). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/index.html>